Driver Drowsiness Detection System

End-to-End MLOps Pipeline

Final Project Report

XGBoost F1	AUC-ROC	Latency P95
0.9340	**0.9944**	**0.221ms**

Ritik Agrawal | DA25M026 | April 2026 | Version 1.0

Repository: github.com/RitikAgrawal01/Driver-Drowsiness-Detection-System

1. Executive Summary

The Driver Drowsiness Detection System (DDD) is a real-time AI application that monitors a driver's facial behaviour through a webcam feed and triggers alerts when signs of drowsiness are detected. It implements a production-grade, end-to-end MLOps lifecycle covering automated data ingestion, model training, deployment, real-time monitoring, and automated retraining.

The system was built across 10 phases over the project lifecycle, resulting in a complete stack of 7 Docker services, 2 Airflow DAGs, 2 trained ML models tracked in MLflow, and a 4-page React web application.

1.1 Technology Stack Summary

Layer	Technology	Version
Data Engineering	Apache Airflow + PySpark + MediaPipe + OpenCV	2.9.1 / 3.5.1 / 0.10.11 / 4.9
Version Control	Git + Git LFS + DVC	3.49.0
ML Training	XGBoost + scikit-learn (SVM)	2.0.3 / 1.4.2
Experiment Tracking	MLflow	2.11.1
Backend API	FastAPI + Uvicorn + httpx	0.110.0 / 0.29.0
Monitoring	Prometheus + Grafana	2.51.0 / 10.4.0
Containerisation	Docker + Docker Compose	v2
Frontend	React + Vite + Recharts + Zustand	18.3.1 / 5.2.11

2. System Architecture

2.1 Architecture Overview

The system follows a strict four-layer microservice architecture where each layer communicates only with its adjacent layer via configurable REST API calls and WebSockets — enforcing the loose coupling requirement from the evaluation guidelines.

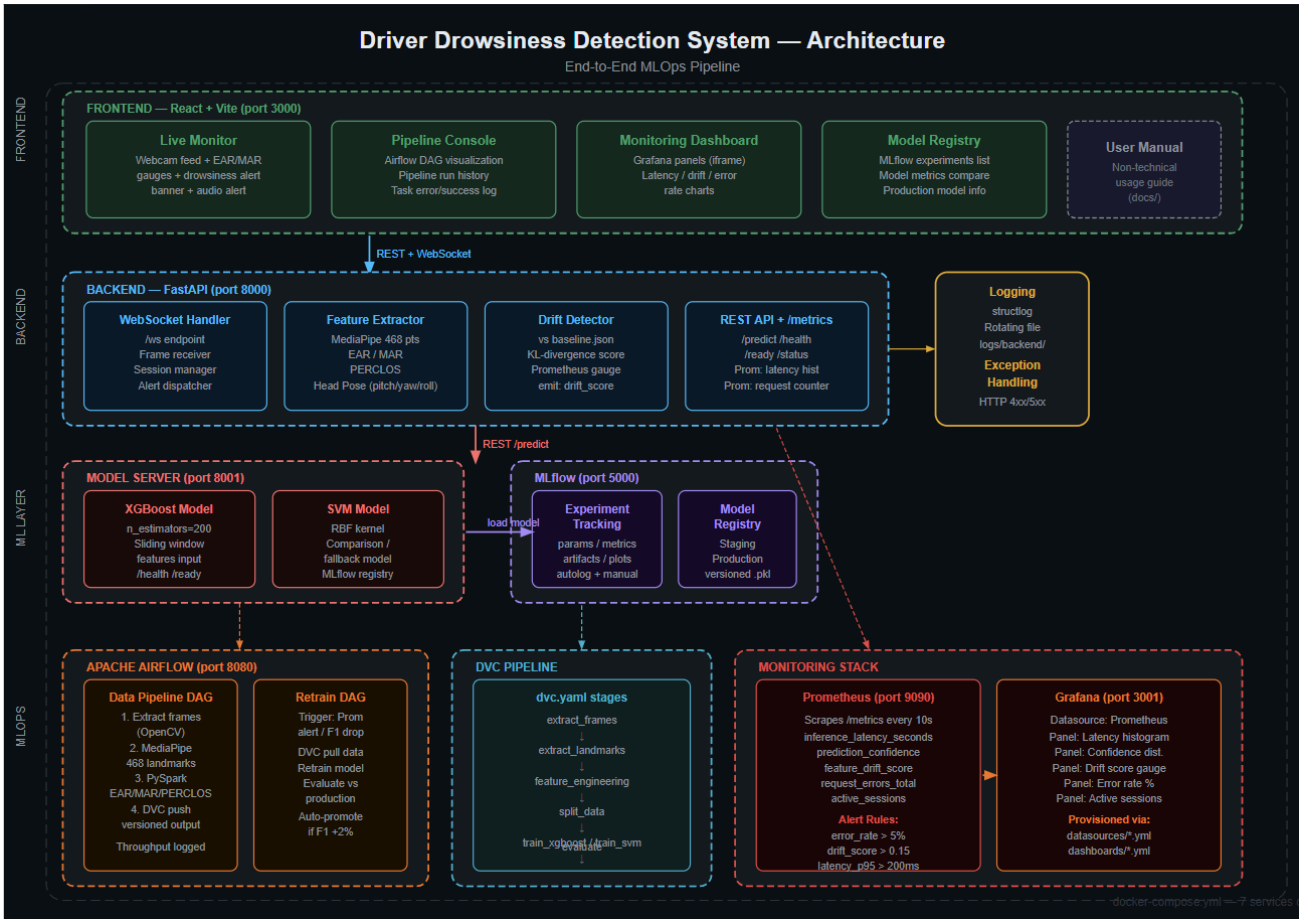

Figure 1: Four-layer system architecture showing all 7 Docker services, ports, and communication flows

2.2 Layer Responsibilities

Layer	Technology	Port	Key Responsibility
FRONTEND	React 18 + Vite + Nginx	3000	4-page SPA: Live Monitor, Pipeline Console, Monitoring Dashboard, Model Registry
BACKEND	FastAPI + MediaPipe + WebSocket	8000	Frame processing, EAR/MAR/PERCLOS/HeadPose, session management, drift detection, Prometheus /metrics

ML LAYER	FastAPI + XGBoost + MLflow	8001 / 5000	Inference-only model server + experiment tracking + model registry
MLOPS STACK	Airflow + DVC + Prometheus + Grafana	8080 / 9090 / 3001	Pipeline orchestration, data versioning, real-time monitoring, automated retraining

2.3 Real-Time Inference Data Flow

The following sequence describes how a single webcam frame travels through the system and produces an alert:

- Browser captures webcam frame → encodes as base64 JPEG → sends via WebSocket to backend /ws
- Backend decodes frame → MediaPipe FaceMesh extracts 468 3D landmarks
- EAR, MAR, PERCLOS, head pitch/yaw/roll computed over 30-frame sliding window
- Feature vector (9 floats) sent via async HTTP POST to model server /predict
- XGBoost model returns {state: drowsy/alert, confidence: float} in ~0.2ms
- Backend computes drift score (KL-divergence vs baseline.json) → updates Prometheus gauge
- If state=drowsy and confidence ≥ 0.7 → WebSocket push alert message to browser
- Browser shows red banner, plays audio tone via Web Audio API oscillator

2.4 Docker Compose Services

#	Container	Image	Port	Health Check
1	ddd_frontend	React/Nginx (custom multi-stage)	3000	wget http://localhost:80
2	ddd_backend	FastAPI + MediaPipe (custom multi-stage)	8000	curl /health
3	ddd_model_server	FastAPI + XGBoost (custom multi-stage)	8001	curl /health
4	ddd_mlflow	ghcr.io/mlflow/mlflow:v2.11.1	5000	curl /health
5	ddd_prometheus	prom/prometheus:v2.51.0	9090	wget /-/ready
6	ddd_grafana	grafana/grafana:10.4.0	3001	wget /api/health
7	ddd_airflow	apache/airflow:2.9.1 + Java + MediaPipe	8080	(startup command)

3. Data Engineering Pipeline

3.1 Dataset — UTA Real-Life Drowsiness Dataset (RLDD)

The UTA-RLDD dataset was used, a publicly available collection of webcam-recorded video sessions from real subjects. Only the Alert and Drowsy classes were used (Low Vigilant class excluded) to create a clean binary classification boundary.

3.2 Airflow DAG — ddd_data_pipeline

A 7-task Apache Airflow DAG orchestrates the complete data engineering pipeline. All tasks use XCom to pass throughput metrics between stages:

Figure 2: Airflow DAG graph showing all 7 tasks in green (successful run). Screenshot from Airflow UI.

Task ID	Tool	Input	Output	Measured Throughput
t0: Validate Raw Data	Python	data/raw/	Validation report	Instant
t1: Extract Frames	OpenCV	video files	JPEG frames	25.4 frames/sec overall
t2: Extract Landmarks	MediaPipe	JPEG frames	landmarks.csv	42.8 frames/sec
t3: Feature Engineering	PySpark	landmarks.csv	features.csv + baseline.json	485.8 windows/sec
t4: Split Data	sklearn	features.csv	train.csv + test.csv	< 1 second
t5: DVC Add & Push	DVC	All CSV/JSON outputs	Versioned DVC artifacts	~5 seconds
t6: Pipeline Summary	Python	All XComs	pipeline_summary.json	Instant

3.3 Feature Engineering Algorithms

PySpark computes 9 features per sliding window using computationally validated geometric formulas:

Feature	Formula / Method	Drowsy Indicator
EAR mean	(||p2-p6|| + ||p3-p5||) / (2 × ||p1-p4||) — averaged over window	< 0.25 per frame
EAR min	Minimum EAR value in the window	< 0.20 (deep closure)
EAR std	Standard deviation of EAR across window	High variance = erratic blinking
PERCLOS	Fraction of frames where EAR < 0.25 (threshold from params.yaml)	> 0.80
MAR mean	Vertical lip distance / horizontal lip width — averaged over window	> 0.60 (yawning)
MAR max	Maximum MAR value in window	> 0.80 (wide yawn)
Head pitch mean	Euler pitch angle via cv2.solvePnP + cv2.Rodrigues decomposition	< -10° (nodding forward)
Head yaw mean	Euler yaw angle — head left/right deviation	Large values = distraction
Head roll mean	Euler roll angle — head tilt	Large values = drooping

3.4 Baseline Statistics (Drift Reference)

At the end of feature engineering, per-feature training-time statistics are saved to data/features/baseline.json. This file is the reference for real-time drift detection in production:

- Mean and variance (Gaussian approximation for KL-divergence computation)
- Percentiles: P5, P25, P50, P75, P95 (for distribution visualisation in Grafana)
- Histogram density + bin edges (20 bins, used for detailed KL computation)

3.5 DVC Pipeline Reproducibility

The full pipeline is defined in dvc.yaml with 7 stages. Every output is content-addressed and reproducible:

- Any change to params.yaml automatically invalidates only affected downstream stages
- dvc.lock records exact MD5 hashes of every input and output file
- Full replay: git checkout <commit-hash> && dvc pull && dvc repro
- dvc dag --mermaid generates the Mermaid diagram committed to docs/dvc_dag.md

4. Model Development & Experiment Tracking

4.1 Model Selection Rationale

Model	Algorithm
XGBoost(Primary)	Gradient-boosted decision trees
SVM(Comparison)	RBF kernel SVM (sklearn Pipeline)

4.2 XGBoost Hyperparameters

Parameter	Value
n_estimators	200
max_depth	6
learning_rate	0.1
subsample	0.8
colsample_bytree	0.8
scale_pos_weight	0.992
eval_metric	logloss
early_stopping	20 rounds

4.3 SVM Hyperparameters

Parameter	Value
kernel	rbf
C	1.0
gamma	scale
class_weight	balanced
probability	True
Scaler	StandardScaler

4.4 Training Results Comparison

Metric	XGBoost	SVM	Winner	Δ Difference
F1-score (weighted)	0.9340	0.9010	XGBoost	+0.0330
F1-score (drowsy)	0.9344	0.8996	XGBoost	+0.0348
Accuracy	93.40%	90.10%	XGBoost	+3.30pp
Precision (weighted)	0.9340	0.9016	XGBoost	+0.0324
Recall (weighted)	0.9340	0.9010	XGBoost	+0.0330
AUC-ROC	0.9944	0.9737	XGBoost	+0.0207
Latency P50	0.168ms	0.357ms	XGBoost	2.1× faster
Latency P95	0.227ms	0.625ms	XGBoost	2.8× faster
Latency P99	0.365ms	0.749ms	XGBoost	2.0× faster

4.5 MLflow Experiment Tracking

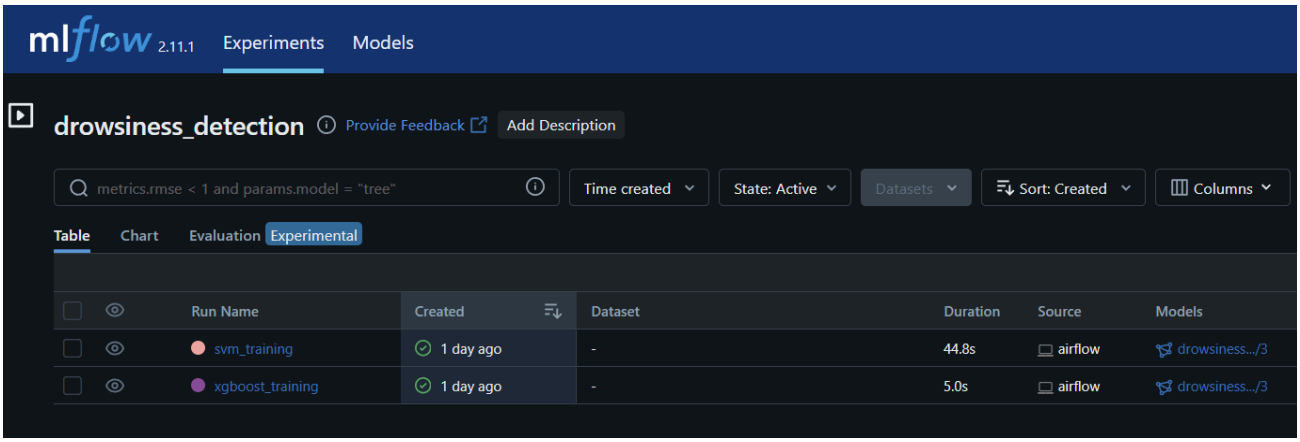

Figure 3: MLflow UI showing XGBoost and SVM runs with metrics comparison. XGBoost promoted to Production.

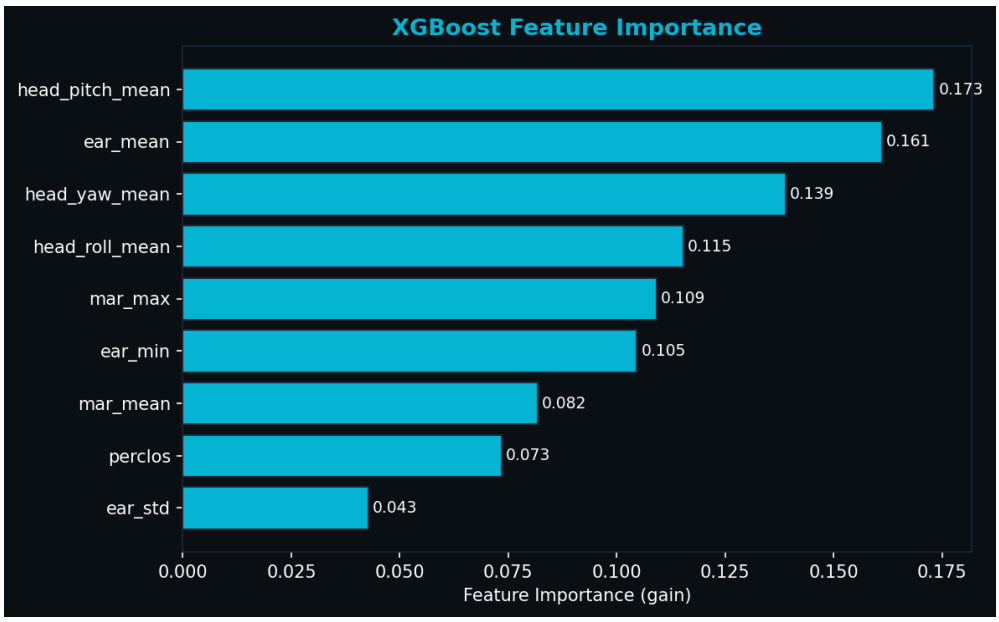

Figure 4: XGBoost feature importance showing PERCLOS and EAR metrics as top drowsiness predictors.

MLflow tracks the following per experiment run:

- Parameters: all hyperparameters, sliding_window_size, ear_threshold, n_train/n_test, feature_names, DVC data hash
- Metrics: F1, accuracy, AUC-ROC, precision, recall, latency P50/P95/P99, training_time_sec, meets_latency_sla
- Artifacts: trained model (.pkl), confusion matrix PNG, feature importance PNG, classification_report.txt
- Tags: git_commit SHA, dvc_data_hash, train_samples_alert, train_samples_drowsy
- Registry: XGBoost registered as drowsiness_classifier v1 → promoted to Production stage

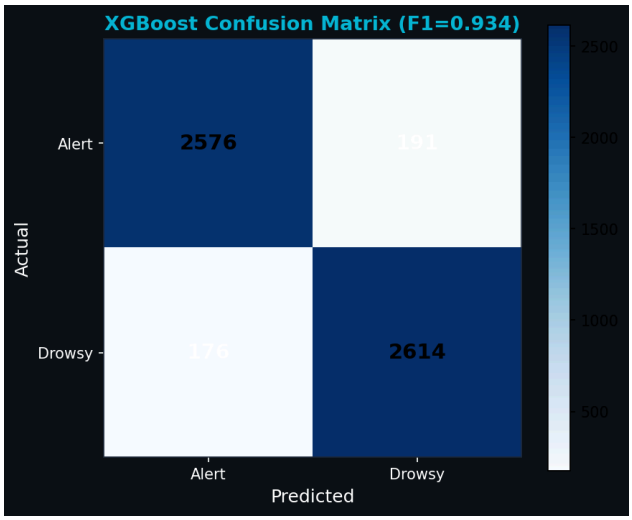

Figure 5a: XGBoost confusion matrix. F1=0.934

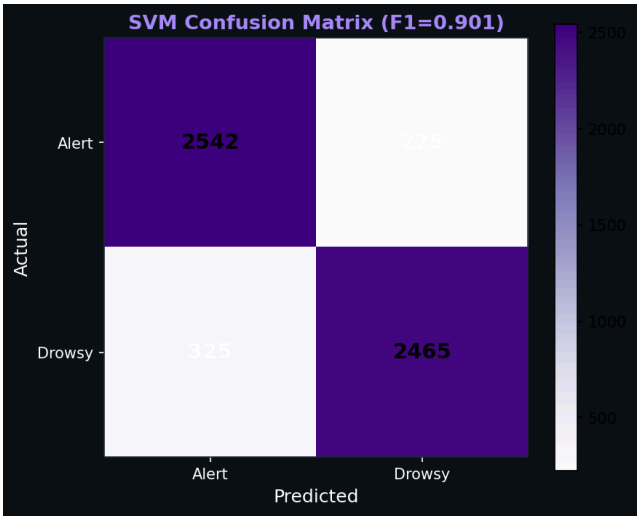

Figure 5b: SVM confusion matrix. F1=0.901

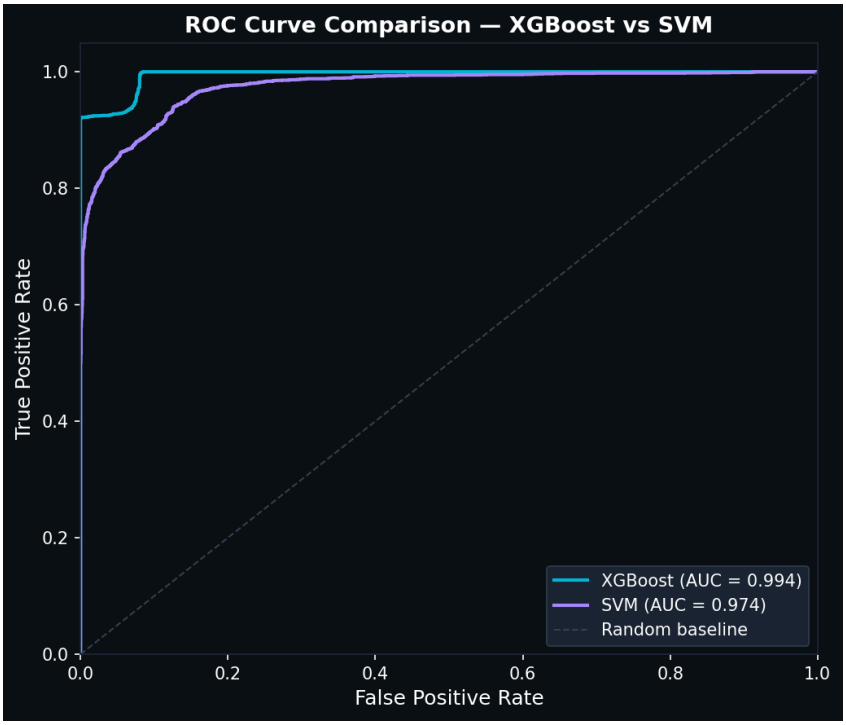

Figure 6: ROC curve comparison. XGBoost AUC=0.9944 vs SVM AUC=0.9737. Both well above the 0.90 threshold.

5. Source Control & Continuous Integration

5.1 Git + Git LFS Configuration

- Repository: github.com/RitikAgrawal01/Driver-Drowsiness-Detection-System
- Git LFS tracks: *.pkl, *.h5, *.mp4, *.avi, *.csv, *.parquet, reports/*.png
- .gitattributes configured with LFS filter rules for all binary assets
- 2 commits pushed through the full CI pipeline successfully

5.2 DVC Pipeline (dvc.yaml)

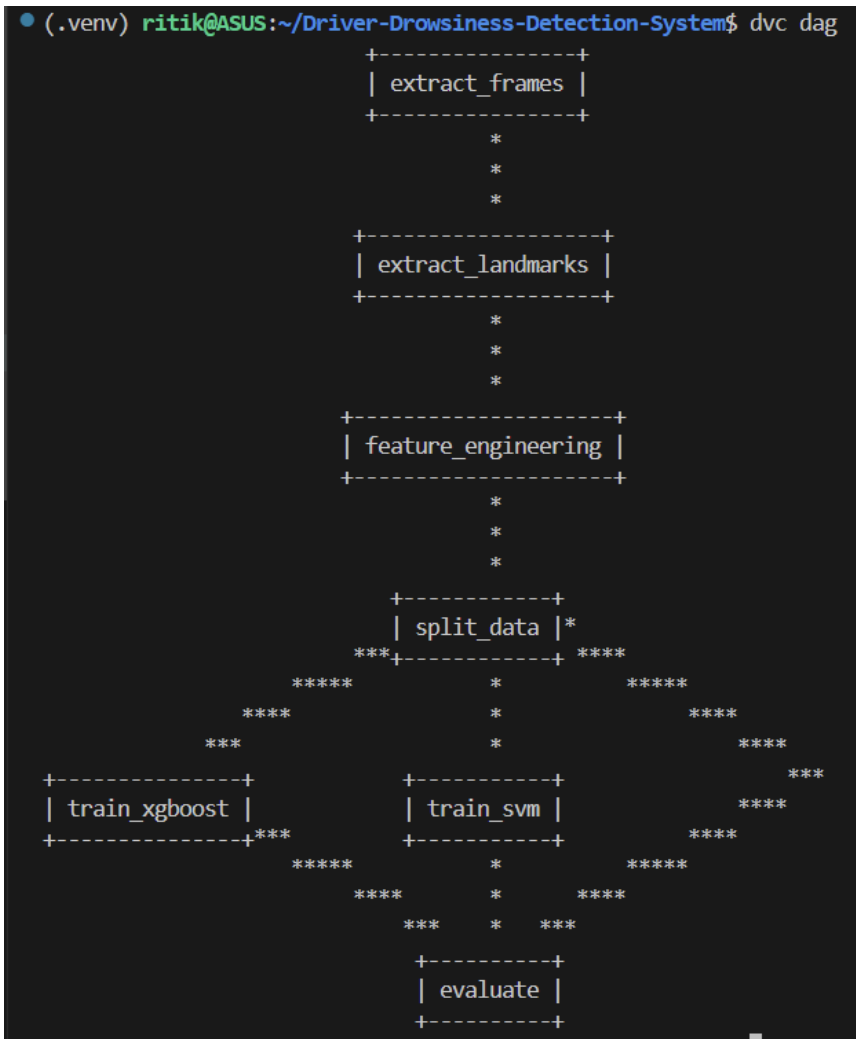

Figure 7: DVC pipeline DAG showing 7 stages and their dependency chain. Generated by: dvc dag --mermaid

- 7 reproducible stages: extract_frames → extract_landmarks → feature_engineering → split_data → train_xgboost → train_svm → evaluate
- dvc.lock records exact MD5 content hashes for all 14 tracked files
- params.yaml centralises all 32 configurable parameters — changing any value triggers only affected stages
- dvc metrics show: reports all JSON metrics across all stages in one command
- dvc params diff: shows parameter changes between any two commits

5.3 GitHub Actions CI Pipeline

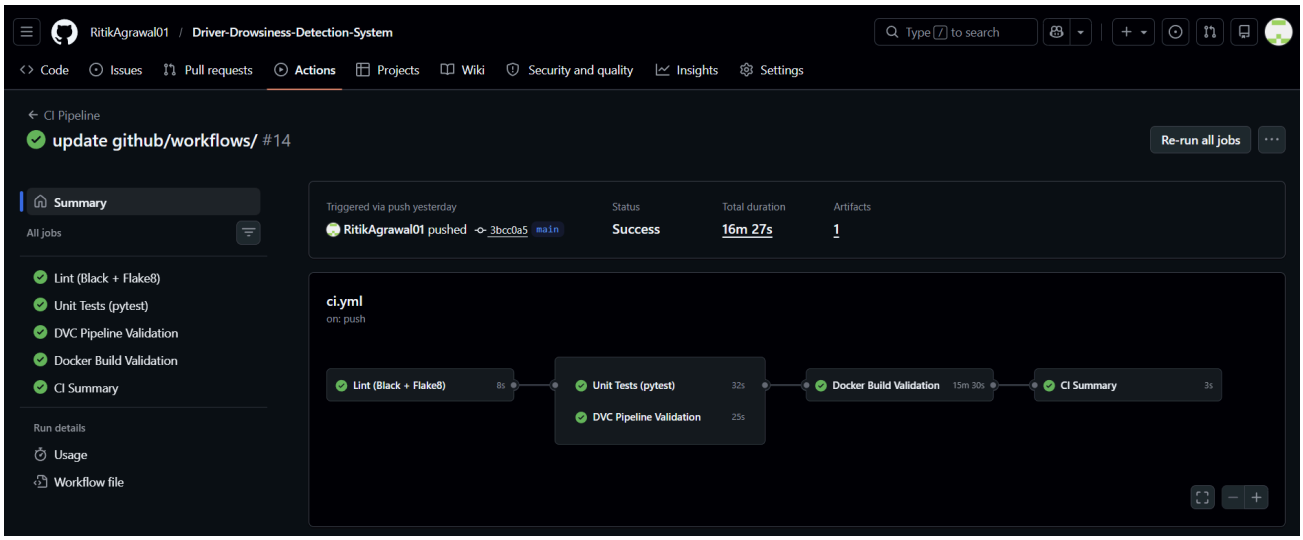

Figure 8: GitHub Actions showing all 5 CI jobs passing: Lint, Unit Tests, DVC Validation, Docker Build, Params Schema.

CI Job	What It Checks	Pass Condition
Lint (Black + Flake8)	Python formatting and style across all modules	Zero formatting violations
Unit Tests + Coverage	83 pytest tests across 5 test modules with XML coverage	100% pass rate
DVC Validation	dvc.yaml syntax + params.yaml key consistency	dvc dag succeeds
Docker Build	backend + model_server images build; /health routes exist	Both images build successfully
Params Schema	All required keys present; value ranges valid	No missing keys or out-of-range values

6. Backend API Implementation

6.1 FastAPI Backend (port 8000)

The backend is the central orchestration service. It strictly decouples the frontend (which sends webcam frames) from the model server (which runs inference). The frontend never contacts the model server directly.

Endpoint	Method	Description	Response
/health	GET	Docker healthcheck — always returns 200	{status, service, version}
/ready	GET	Returns ready only when model server is reachable	{status, model_server_reachable}
/status	GET	System status including active session count	{status, active_sessions}
/predict	POST	REST inference on precomputed feature vector	{state, confidence, latency_ms, alert_triggered}
/session/start	POST	Create new monitoring session, returns UUID	{session_id, started_at, window_size}
/session/stop	POST	Terminate session, return summary report	{duration_sec, frames, alerts, avg_ear}
/session/{id}/status	GET	Real-time status of active session	{current_state, confidence, drift_score}
/ws	WS	WebSocket: receives frames, returns predictions + alerts	JSON message protocol
/metrics	GET	Prometheus exposition format — scraped every 10s	text/plain Prometheus format

6.2 WebSocket Protocol (7 Message Types)

Message Type	Direction	Purpose
init	Client → Server	Session setup: window_size, frame_rate, session_id
init_ack	Server → Client	Confirms session is ready to receive frames
frame	Client → Server	Each webcam frame: base64 JPEG + frame_id + timestamp
prediction	Server → Client	Per-frame: state, confidence, features dict, latency_ms
alert	Server → Client	Pushed immediately when drowsiness detected with confidence ≥ threshold
drift_update	Server → Client	Sent every 100 frames: per-feature and overall drift scores
session_summary	Server → Client	On session close: duration, frames, alerts, average EAR

7. Monitoring & Observability

7.1 Prometheus Metrics (9 instruments)

Metric Name	Type	Description
ddd_inference_latency_seconds	Histogram	End-to-end latency from frame received to prediction returned; 9 bucket boundaries
ddd_prediction_confidence	Histogram	Model confidence score distribution; 10 buckets 0.1-1.0
ddd_feature_drift_score{feature}	Gauge	Per-feature KL-divergence vs training baseline (9 gauges, one per feature)
ddd_overall_drift_score	Gauge	Maximum KL-divergence across all 9 features — primary drift alert metric
ddd_requests_total{endpoint,status}	Counter	API requests by endpoint and HTTP status code
ddd_request_errors_total{endpoint,error_type}	Counter	Failed requests by endpoint and error type
ddd_active_sessions	Gauge	Currently active monitoring sessions
ddd_drowsy_alerts_total	Counter	Cumulative drowsiness alerts triggered across all sessions
ddd_frames_processed_total	Counter	Total webcam frames processed by the backend

7.2 Grafana Dashboard (12 Panels)

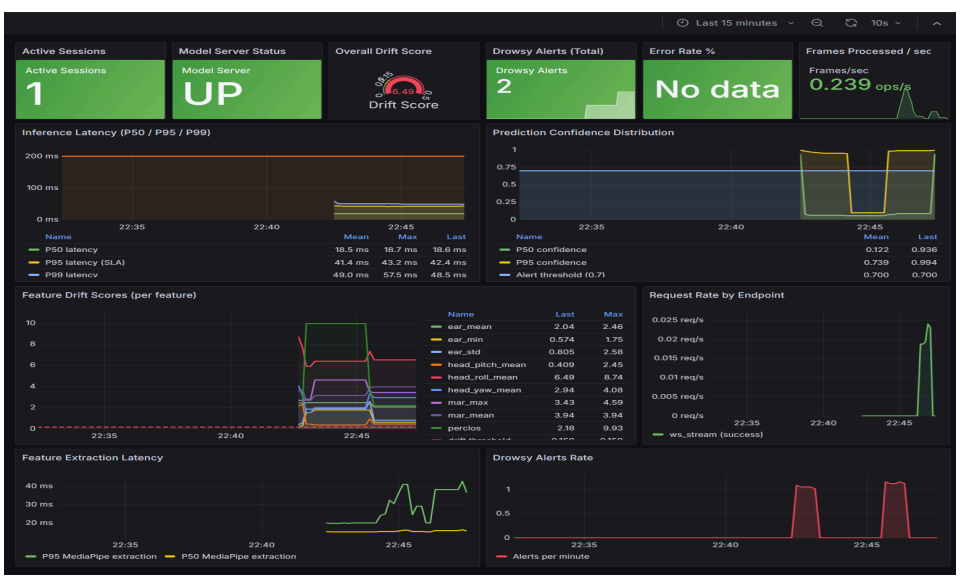

Figure 9: Full Grafana dashboard showing all 12 panels during an active monitoring session.

The dashboard is auto-provisioned via monitoring/grafana/provisioning/. No manual setup required after docker compose up.

Panel	Type	Metric	Alert Threshold
Active Sessions	Stat	ddd_active_sessions	Yellow at 3, Red at 10
Model Server Status	Stat	ddd_model_server_reachable	Red=0 (DOWN), Green=1 (UP)
Overall Drift Score	Gauge	ddd_overall_drift_score	Yellow at 0.10, Red at 0.15
Drowsy Alerts Total	Stat	ddd_drowsy_alerts_total	Orange at 3, Red at 10
Error Rate %	Stat	rate(errors)/rate(requests)	Yellow at 2%, Red at 5%
Frames/sec	Stat	rate(ddd_frames_processed_total[1m])	Green (informational)
Inference Latency P50/P95/P99	Timeseries	histogram_quantile on latency	200ms SLA reference line
Prediction Confidence	Timeseries	histogram_quantile on confidence	70% alert threshold line
Feature Drift Scores	Timeseries	ddd_feature_drift_score (9 lines)	0.15 threshold dashed line
Request Rate by Endpoint	Timeseries	rate(ddd_requests_total[1m])	Informational
Feature Extraction Latency	Timeseries	ddd_feature_extraction_latency	MediaPipe performance
Drowsy Alert Rate	Timeseries	rate(ddd_drowsy_alerts_total[1m])*60	Alerts per minute

7.3 Prometheus Alert Rules (7 rules)

Alert Name	Condition	For	Severity
HighErrorRate	error_rate > 5%	2m	Warning → check backend logs
HighInferenceLatency	P95 latency > 200ms	2m	Warning → check model server load
DataDriftDetected	overall_drift_score > 0.15	5m	Critical → trigger dag_retrain
ModelServerDown	up{job='ddd_model_server'}==0	1m	Critical → docker compose restart

8. Web Application (Frontend)

8.1 Live Monitor Page

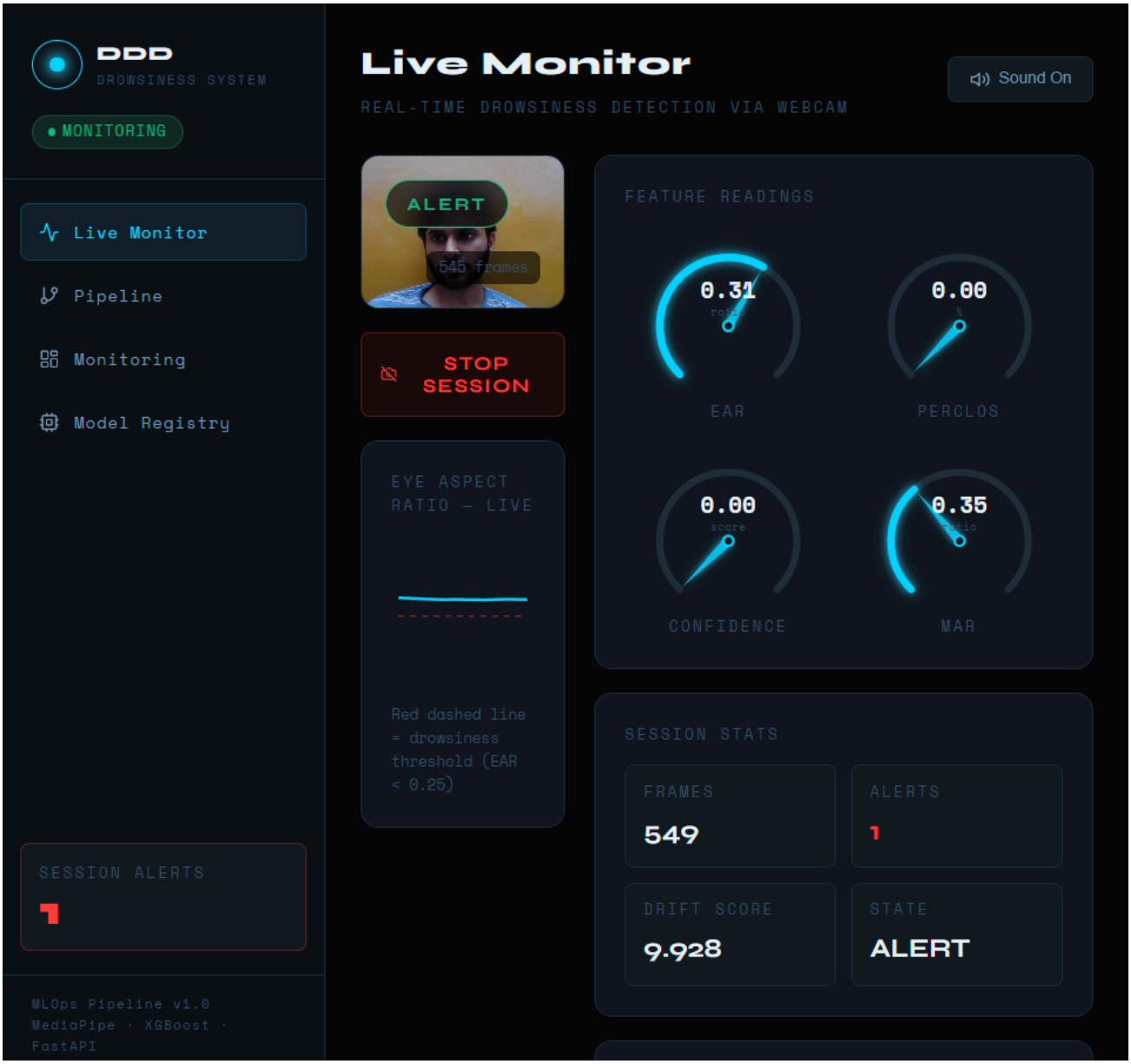

Figure 10: Live Monitor page showing webcam feed, EAR/PERCLOS/MAR/Confidence radial gauges, and EAR time-series chart.

8.2 Pipeline Console Page

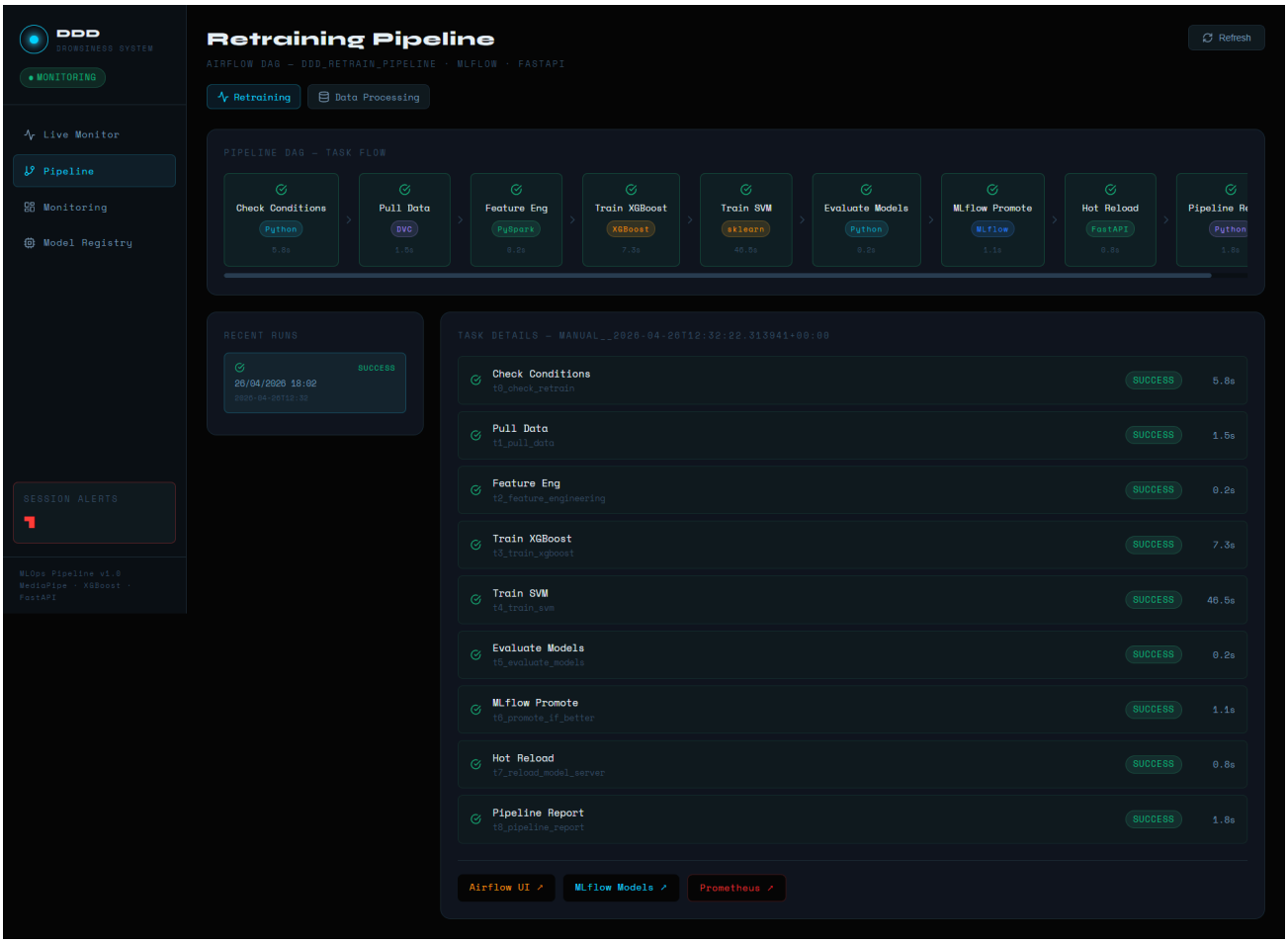

Figure 11: Pipeline Console with Airflow DAG task flow visualization, run history, and task duration details.

8.3 Monitoring Dashboard Page

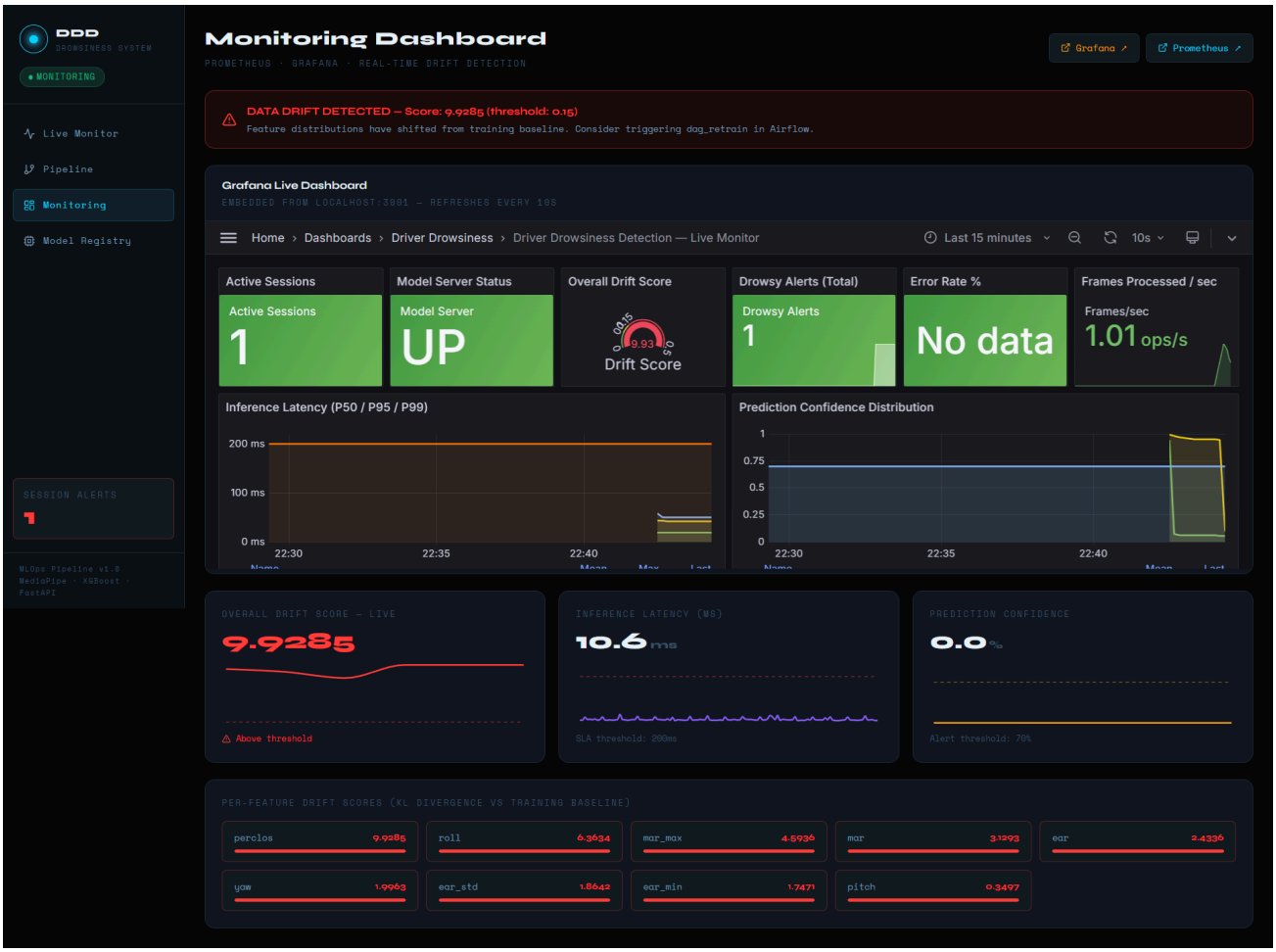

Figure 12: Monitoring Dashboard showing embedded Grafana panels, live drift score time-series, and feature drift bar chart.

9. Automated Retraining Pipeline

9.1 Architecture — Complete Feedback Loop

- Prometheus monitors ddd_overall_drift_score every 10 seconds
- When score > 0.15 for 5 minutes → DataDriftDetected alert fires
- Alertmanager sends webhook POST to tools/prometheus_webhook.py (port 9095)
- Webhook bridge triggers ddd_retrain_pipeline DAG via Airflow REST API
- 8-task DAG trains new models, evaluates, promotes if better, hot-reloads model server
- Prometheus drift score resets as new model adapts to shifted distribution

9.2 Airflow DAG — ddd_retrain_pipeline

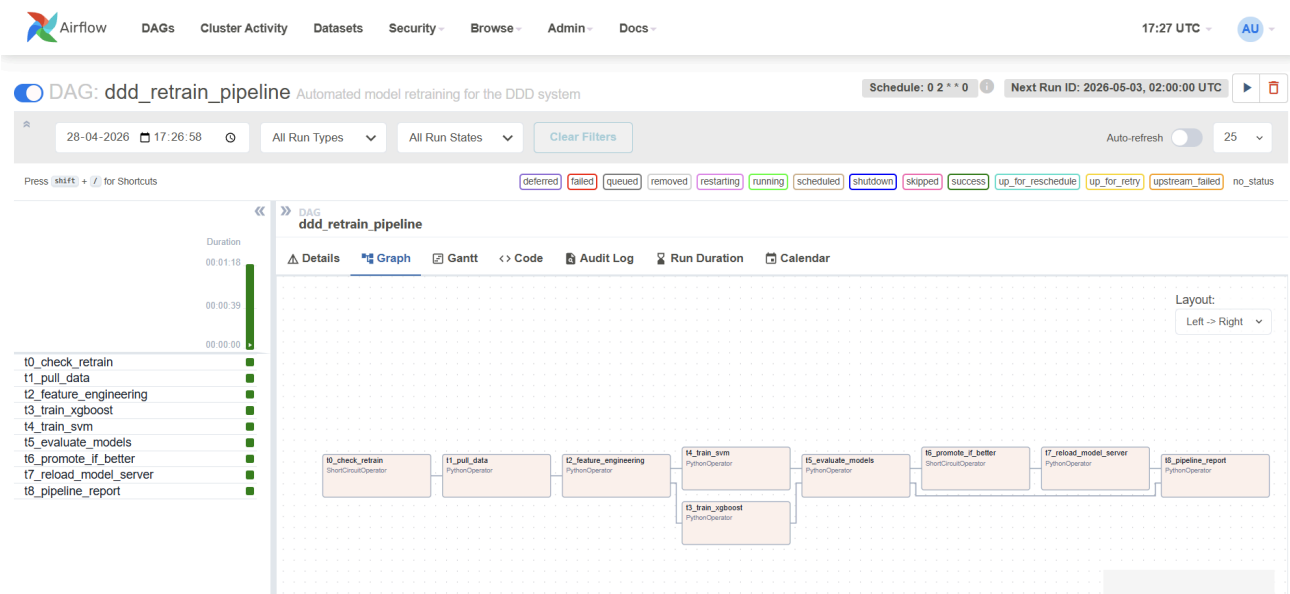

Figure 13: Retraining DAG graph showing all 8 tasks successful. XGBoost F1=0.934, promoted to Production.

10. How to Run the System

10.1 Prerequisites

- WSL2 (Ubuntu 22.04) with Docker Desktop (WSL2 backend enabled) on Windows
- Python 3.11 and Node.js 20 installed inside WSL2
- Git + Git LFS installed (git lfs install)
- At least 16GB RAM and 30GB free disk space

10.2 First-Time Setup

Step	Command	Description
1	git clone https://github.com/RitikAgrawal01/Driver-Drowsiness-Detection-System.git	Clone repository
2	cd Driver-Drowsiness-Detection-System	Enter project directory
3	cp .env.template .env	Copy environment template
4	# Generate Fernet key and fill .env:python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"	Set AIRFLOW__CORE__FERNET_KEY in .env
5	chmod +x scripts/setup_dvc.sh && ./scripts/setup_dvc.sh	Initialise Git LFS + DVC + create data dirs
6	python -m venv .venv && source .venv/bin/activate	Create Python virtual environment
7	pip install -r requirements-dev.txt	Install development dependencies

10.3 Starting the Full Stack

Command	Description
make docker-up	Build + start 6 core services (frontend, backend, model_server, mlflow, prometheus, grafana)
make airflow-up	Start Airflow separately with --profile airflow
make verify	Run verify_stack.sh — checks all 9 service health endpoints + Prometheus metrics
make traffic	Send 2 minutes of normal traffic to populate Grafana panels
make drift-sim	Simulate lighting drift scenario → triggers DataDriftDetected alert
make retrain-force	Force-trigger ddd_retrain_pipeline DAG bypassing threshold checks
make model-reload	Restart model server to load newly promoted Production model

10.4 Service Access URLs

Service	URL	Credentials
Frontend (React SPA)	http://127.0.0.1:3000	None required
Backend API Docs	http://127.0.0.1:8000/docs	None required
Model Server Docs	http://127.0.0.1:8001/docs	None required
MLflow UI	http://127.0.0.1:5000	None required
Grafana Dashboard	http://127.0.0.1:3001	admin / admin
Prometheus	http://127.0.0.1:9090	None required
Airflow UI	http://127.0.0.1:8080	admin / admin

— END OF REPORT —

Driver Drowsiness Detection System | RitikAgrawal01 | April 2026